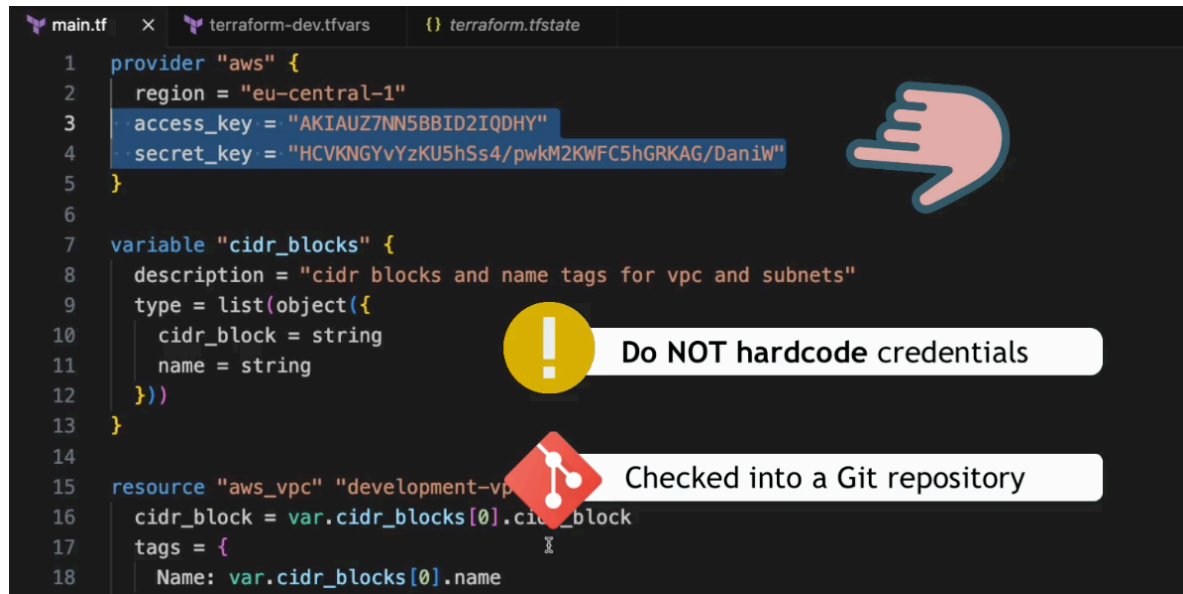


Module 12 - chapter 10

Environment Variables in Terraform

Writing the AWS credentials is not a good security practice

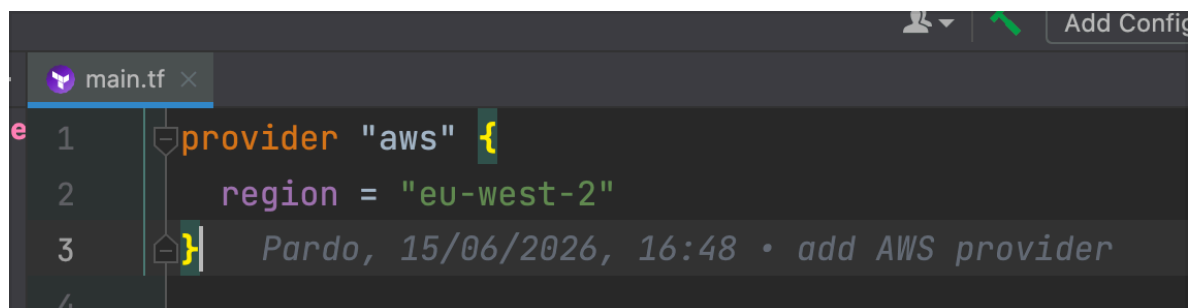


```
1 provider "aws" {
2   region = "eu-central-1"
3   access_key = "AKIAUZ7NN5BBID2IQDHY"
4   secret_key = "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"
5 }
6
7 variable "cidr_blocks" {
8   description = "cidr blocks and name tags for vpc and subnets"
9   type = list(object({
10     cidr_block = string
11     name = string
12   }))
13 }
14
15 resource "aws_vpc" "development-vp" {
16   cidr_block = var.cidr_blocks[0].cidr_block
17   tags = {
18     Name: var.cidr_blocks[0].name
```

We might end up pushing them to a Git repo and now other ppl can see them



So we won't use them in the main.tf so they re now removed



```
1 provider "aws" {
2   region = "eu-west-2"
3 }
4
```

Pardo, 15/06/2026, 16:48 • add AWS provider

And there are two ways to set these credentials so TF can pick it up (I have already used env vars see notes "Shared AWS 'admin' acces key and secret via github")

1. Setting up the credentials as env vars

In the terminal I execute:

```
export AWS_ACCESS_KEY_ID="your-key"
export AWS_SECRET_ACCESS_KEY="your-secret"
```

I already did the above and TF has been working.

This option works only for the session of the terminal where I executed the export.

If I want the credentials to be available globally then I need to make sure that I had run

-> aws configure

So when I execute -> terraform apply it access the global credentials, as seen below:

```
→ terraform git:(Module_12/Terraform_chapter-10) × terraform apply -var-file terraform-dev.tfvars

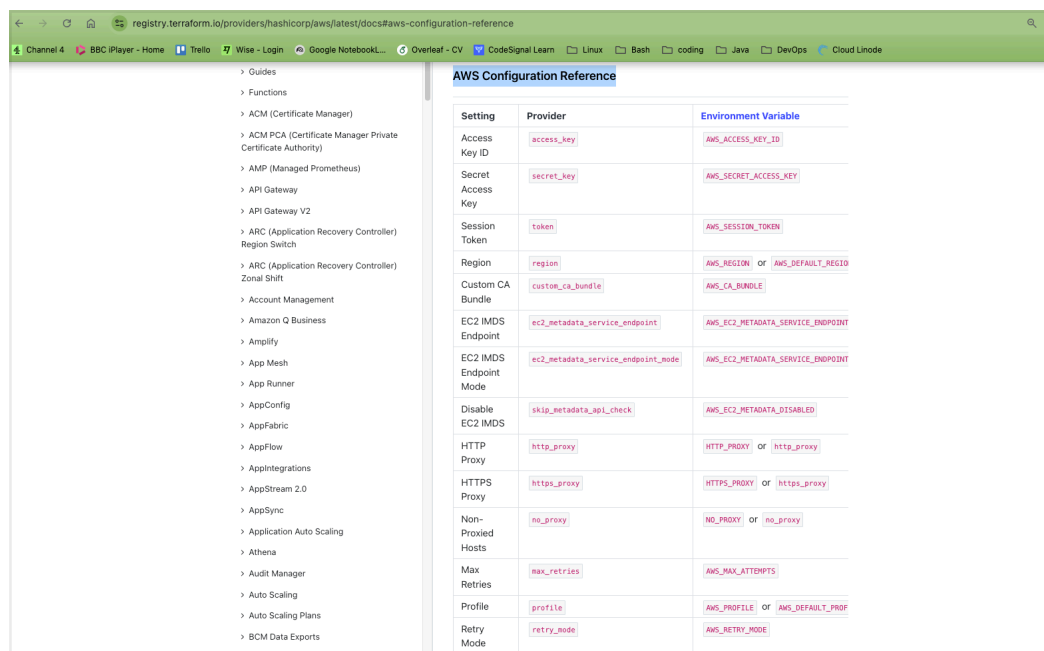
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
  + create

Terraform will perform the following actions:

# aws_subnet.dev-subnet-1 will be created
+ resource "aws_subnet" "dev-subnet-1" {
  + arn                                = (known after apply)
  + assign_ipv6_address_on_creation    = false
  + availability_zone                  = "eu-west-2a"
  + availability_zone_id                = (known after apply)
  + cidr_block                         = "10.0.10.0/24"
  + enable_dns64                       = false
```

In the documentation of each provider we can find the env vars that can be used for that specific provider. For example for AWS:

<https://registry.terraform.io/providers/hashicorp/aws/latest/docs#aws-configuration-reference>



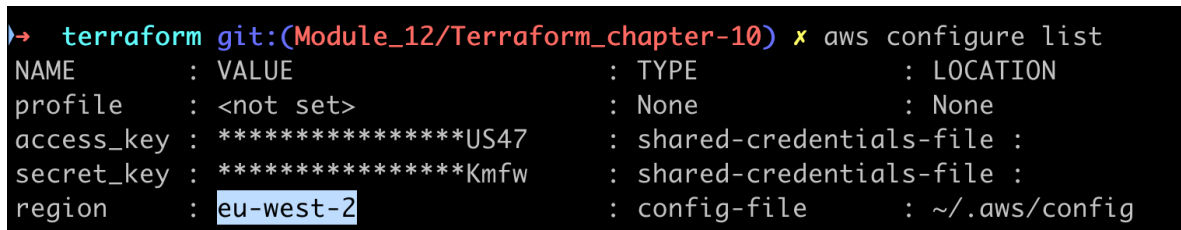
Setting	Provider	Environment Variable
Access Key ID	access_key	AWS_ACCESS_KEY_ID
Secret Access Key	secret_key	AWS_SECRET_ACCESS_KEY
Session Token	token	AWS_SESSION_TOKEN
Region	region	AWS_REGION or AWS_DEFAULT_REGION
Custom CA Bundle	custom_ca_bundle	AWS_CA_BUNDLE
EC2 IMDS Endpoint	ec2_metadata_service_endpoint	AWS_EC2_METADATA_SERVICE_ENDPOINT
EC2 IMDS Endpoint Mode	ec2_metadata_service_endpoint_mode	AWS_EC2_METADATA_SERVICE_ENDPOINT_MODE
Disable EC2 IMDS	skip_metadata_api_check	AWS_EC2_METADATA_DISABLED
HTTP Proxy	http_proxy	HTTP_PROXY or http_proxy
HTTPS Proxy	https_proxy	HTTPS_PROXY or https_proxy
Non-Proxy	no_proxy	NO_PROXY or no_proxy
Max Retries	max_retries	AWS_MAX_ATTEMPTS
Profile	profile	AWS_PROFILE or AWS_DEFAULT_PROFILE
Retry Mode	retry_mode	AWS_RETRY_MODE

In my aws configure I also setup a default region 'eu-west-2' so I actually don't need it in the main.tf file



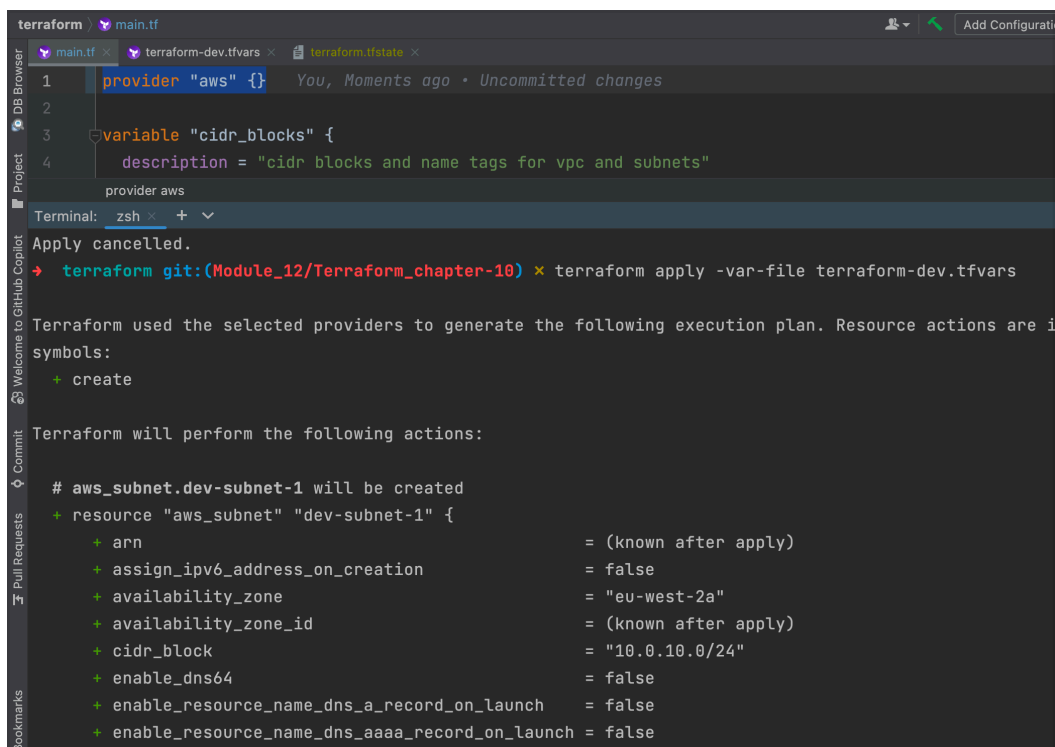
```
1 provider "aws" {
2   # region = "eu-west-2"
3 }
```

-> aws configure list



```
> terraform git:(Module_12/Terraform_chapter-10) x aws configure list
NAME      : VALUE                                : TYPE      : LOCATION
profile   : <not set>                               : None      : None
access_key : *****US47                             : shared-credentials-file :
secret_key : *****Kmfw                             : shared-credentials-file :
region    : eu-west-2                               : config-file  : ~/.aws/config
```

And it works:



```
1 provider "aws" {}
2
3 variable "cidr_blocks" {
4   description = "cidr blocks and name tags for vpc and subnets"
5 }

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_subnet.dev-subnet-1 will be created
+ resource "aws_subnet" "dev-subnet-1" {
  + arn                                = (known after apply)
  + assign_ipv6_address_on_creation    = false
  + availability_zone                  = "eu-west-2a"
  + availability_zone_id                = (known after apply)
  + cidr_block                         = "10.0.10.0/24"
  + enable_dns64                       = false
  + enable_resource_name_dns_a_record_on_launch = false
  + enable_resource_name_dns_aaaa_record_on_launch = false
```

Set variable using TF environment variable

If I want to define my own custom env var in TF I can do it ->
TF_VAR_<any_name>

Example, we want to work in a specific AZ

```

18 resource "aws_subnet" "dev-subnet-1" {
19     vpc_id = aws_vpc.development-vpc.id
20     cidr_block = var.cidr_blocks[1].cidr_block
21     availability_zone = "eu-west-2a"
22     tags = {
23         Name: var.cidr_blocks[1].name
24     }
25 }
26

```

-> export TF_VAR_avail_zone="eu-west-2a"

```

→ terraform git:(Module_12/Terraform_chapter-10) × export TF_VAR_avail_zone="eu-west-2a"

```

TF_VAR -> tells tf this is a global variable

avail_zone -> name of the variable that we can reference

So now we go to our main.tf and create the variable

```

main.tf × terraform-dev.tfvars × terraform.tfstate ×
7
8     )))
9 }
10
11 variable "avail_zone" {}
12
13 resource "aws_vpc" "development-vpc" {
14     cidr_block = var.cidr_blocks[0].cidr_block

```

And now we can reference it where we need it:

```

19
20 resource "aws_subnet" "dev-subnet-1" {
21     vpc_id = aws_vpc.development-vpc.id
22     cidr_block = var.cidr_blocks[1].cidr_block
23     availability_zone = var.avail_zone
24     tags = {
25         Name: var.cidr_blocks[1].name
26     }
27 }

```

-> terraform apply -var-file terraform-dev.tfvars

```

+ terraform git:(Module_12/Terraform_chapter-10) x terraform apply -var-file terraform-dev.tfvars
aws_vpc.development-vpc: Refreshing state... [id=vpc-074d48603ae8fb1b1]
aws_subnet.dev-subnet-1: Refreshing state... [id=subnet-08c867e0fd287d891]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
+ terraform git:(Module_12/Terraform_chapter-10) x

```

We used the same value for AZ, so now let's use a different value "eu-west-2b"

```

+ terraform git:(Module_12/Terraform_chapter-10) x export TF_VAR_avail_zone="eu-west-2b"
+ terraform git:(Module_12/Terraform_chapter-10) x

```

-> terraform apply -var-file terraform-dev.tfvars

```

+ terraform git:(Module_12/Terraform_chapter-10) x terraform apply -var-file terraform-dev.tfvars
aws_vpc.development-vpc: Refreshing state... [id=vpc-074d48603ae8fb1b1]
aws_subnet.dev-subnet-1: Refreshing state... [id=subnet-08c867e0fd287d891]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

# aws_subnet.dev-subnet-1 must be replaced
-/+ resource "aws_subnet" "dev-subnet-1" {
    ~ arn                                = "arn:aws:ec2:eu-west-2:706976333940:subnet/subnet-08c867e0fd287d891" -> (known a
fter apply)
    ~ availability_zone                  = "eu-west-2a" -> "eu-west-2b" # forces replacement
    ~ availability_zone_id               = "euw2-az2" -> (known after apply)
    - enable_lni_at_device_index         = 0 -> null
    ~ id                                = "subnet-08c867e0fd287d891" -> (known after apply)
    + ipv6_cidr_block                    = (known after apply)
    + ipv6_cidr_block_association_id     = (known after apply)
}

```

We can see that the AZ value will be replaced so the whole resource will be replaced

```

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

Enter a value: yes

aws_subnet.dev-subnet-1: Destroying... [id=subnet-08c867e0fd287d891]
aws_subnet.dev-subnet-1: Destruction complete after 1s
aws_subnet.dev-subnet-1: Creating...
aws_subnet.dev-subnet-1: Creation complete after 1s [id=subnet-0eec51f2c958da8ea]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
→ terraform git:(Module_12/Terraform_chapter-10) ×

```

Let's check AWS UI

The screenshot shows the AWS Management Console interface for Subnets. At the top, there's a header 'Subnets (1/4)' with a search bar and a 'Create subnet' button. Below this is a table listing several subnets. The 'dev-subnet' is highlighted with a blue selection bar. Below the table, the details for 'subnet-0eec51f2c958da8ea / dev-subnet' are displayed. The details are organized into four columns: Subnet ID, Subnet ARN, State, and Block Public Access. The Subnet ID is 'subnet-0eec51f2c958da8ea', the Subnet ARN is 'arn:aws:ec2:eu-west-2:706976333940:subnet/subnet-0eec51f2c958da8ea', the State is 'Available', and Block Public Access is 'Off'. Other details include IPv4 CIDR '10.0.10.0/24', Availability Zone 'euw2-az3 (eu-west-2b)', Network ACL, Network border group 'eu-west-2', and VPC 'vpc-074d48603ae8fb1b1 | dev-vpc'.

Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR	IPv6 ...	IPv6 ...	Avail...	Availability Zone
-	subnet-03da2ce0b3dba523	Available	vpc-03677c887cc2f715	Off	172.31.0.0/20	-	-	4090	euw2-az1 (eu-west-2c)
☒	dev-subnet subnet-0eec51f2c958da8ea	Available	vpc-074d48603ae8fb1b1 dev-vpc	Off	10.0.10.0/24	-	-	251	euw2-az3 (eu-west-2b)
-	subnet-092c85ea013bc240	Available	vpc-03677c887cc2f715	Off	172.31.16.0...	-	-	4091	euw2-az2 (eu-west-2a)
-	subnet-051c9624093279c7e	Available	vpc-03677c887cc2f715	Off	172.31.32.0...	-	-	4090	euw2-az3 (eu-west-2b)

subnet-0eec51f2c958da8ea / dev-subnet

Details | Flow logs | Route table | Network ACL | CIDR reservations | Sharing | Tags

Details

Subnet ID
subnet-0eec51f2c958da8ea

IPv4 CIDR
10.0.10.0/24

Availability Zone
euw2-az3 (eu-west-2b)

Network ACL

Subnet ARN
arn:aws:ec2:eu-west-2:706976333940:subnet/subnet-0eec51f2c958da8ea

Available IPv4 addresses
251

Network border group
eu-west-2

State
Available

IPv6 CIDR
-

VPC
vpc-074d48603ae8fb1b1 | dev-vpc

Auto-assign public IPv4 address

Block Public Access
Off

IPv6 CIDR association ID
-

Route table
rtb-027f79f2e67f3d81

Auto-assign IPv6 address

We can see the new subnet has AZ 2b instead of 2a.